Lab 04 — Questions

Question 1 [Diagnosis]

A developer runs `podman build -t api:1.0 .` from `~/projects/api/`. Their Dockerfile contains `COPY ../common/config.yml /app/`, and the build fails with `possible escaping context directory error`. Explain why it fails, and why neither `-f`, nor an absolute path, nor `sudo` will make any difference. What is the correct fix?

Question 2 [Analysis]

Under Docker, building an Angular project takes 4 minutes, 3 min 20 s of which show up as `transferring context`. The project folder contains `node_modules/` (900 MB) and `.git/` (200 MB). A colleague switches to Podman, watches the build drop to 40 seconds, and concludes that `.dockerignore` is no longer needed. Explain what Docker was doing, what changed with Podman, and why your colleague is wrong — name the **second** risk, the one that has nothing to do with speed.

Question 3 [Understanding]

A Dockerfile contains `EXPOSE 8080`. The developer runs `podman run -d my-api:1.0`, finds that `curl http://localhost:8080` gets no answer, and concludes the image is broken. Is that conclusion right? Explain what `EXPOSE` actually does.

Question 4 [Analysis]

Compare these two Dockerfiles. What exactly does each one do, and which is correct for an API image?

```
# A
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
RUN java -jar /app/api.jar
```

```
# B
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
CMD ["java","-jar","/app/api.jar"]
```

Question 5 [Analysis]

Here are two images. For each one, say what `podman run img` and `podman run img --debug` produce.

```
# A
ENTRYPOINT ["java","-jar","/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
# B
CMD ["java","-jar","/app/api.jar","--spring.profiles.active=prod"]
```

Then say which of the two still lets you run `podman run img sh` for debugging, and how to get a shell with the other one.

Question 6 [Diagnosis]

A team complains that every redeployment takes ten seconds longer than expected, that Podman prints `resorting to SIGKILL` each time, and that Spring Boot never runs its *shutdown hooks*. The Dockerfile ends with:

```
CMD java -jar /app/api.jar
```

Diagnose the problem, fix it, and explain why this single line is enough to cause the symptom — and why the team does **not** see it on their Alpine-based test image.

Question 7 [Analysis]

A Dockerfile needs the `JAVA_OPTS` variable at start-up:

```
ENV JAVA_OPTS="-Xmx512m"
ENTRYPOINT ["java","$JAVA_OPTS","-jar","/app/api.jar"]
```

The container fails with `Unrecognized option: $JAVA_OPTS`. Explain the failure, then give **two** possible fixes and state what each one costs.

Question 8 [Understanding]

A developer passes the database password at build time — `podman build --build-arg DB_PASSWORD=Secr3t! -t api:1.0 .` — arguing that an `ARG` does not persist in the image. Is the password safe? Justify your answer and give the command that proves it.

Question 9 [Analysis]

A Maven Dockerfile is written like this:

```
COPY . /app
WORKDIR /app
RUN mvn -q package -DskipTests
```

Every build takes 6 minutes, even when only a single `.java` file changed. Rewrite the instructions in the right order, explain the cache mechanism that makes your version faster, and say which kind of build will stay slow anyway.

Question 10 [Diagnosis]

```
RUN apt-get update
RUN apt-get install -y curl vim
RUN rm -rf /var/lib/apt/lists/*
```

Point out **three** distinct problems with these three lines, and give the correct version.

Question 11 [Analysis]

A developer changes only the last `COPY` in their Dockerfile and watches the build print `--> Using cache` for the first eight steps, then rebuild the last two. The next day, they insert an `ENV` variable

in **third** position, and the whole build starts from scratch. Explain both behaviors with one and the same rule.

Question 12 [Understanding]

`COPY` and `ADD` look interchangeable. Give two behaviors specific to `ADD`, explain why the official recommendation is to use `COPY`, and name the one case where `ADD` is still justified.

Question 13 [Analysis]

A Dockerfile puts `USER 1000:1000` right after the `FROM`, before the `COPY` and `RUN apt-get install` instructions. The build fails with `Permission denied`. Explain the failure and say where `USER` belongs in a well-written Dockerfile. Then: in rootless mode, `podman top` shows `USER 1000` and `HUSER 100999` for this container. Where does that second number come from, and what is `USER` still good for, given that "root" is not really root anyway?

Question 14 [Analysis]

A colleague claims: "You should keep the number of layers as low as possible, so put everything in a single `RUN`." Discuss: when is this right, and when does the rule hurt build time and transfer size?